# Building, Programming and Controlling a Quadcopter

## 1: Introducing the Project

### 1.1: Introduction and Project plan

Welcome to the report for my CREST silver award project.

This short section is dedicated to itemising my approach to this problem and giving the reader a clear roadmap for it. There will be no detailed discussion of ideas in this section, but the timeline should give a good understanding of how I aim to build, program and control a quadcopter. It is as follows:

1. Understand the Physics of a Quadcopter (≈1.5 months);
2. Implement a model of this physics in code (≈2.5 months);
3. Decide how to couple a control system to this physics and choose appropriate hardware for its implementation (≈1 months);
4. Verify that the control system works as expected using this mathematical model (continues throughout project);
5. Test any sensors that are needed to give data to the control system (≈2 months);
6. Test the motors outside of the quadcopter assembly (≈2 weeks);
7. Assemble the quadcopter and wire/solder the electronics (only possible after I had all the components (≈1 month);
8. Write a user interface to control the drone remotely over Wi-Fi from a laptop (≈2 weeks);
9. Collect physical data about the drone to feed to the model (≈1 week);
10. Using this computer model, tune the control system and export the values to the drone (ongoing).

This first chapter will be an introduction to some of my reasoning for the approach above, as well as a detailed overview of the different components that I will be using for each stage, and why I chose them. The second chapter will take up most of the rest of the report, and will be a detailed start-to-finish description of everything that went into the project what I learnt along the way.

### 1.2: Inspiration

Quadcopters and the systems that control them are now ubiquitous in our society. Some are sold as toys, some as military equipment, and there are even plans in the future to be able to deliver packages by drone. I have always had a fascination with these flying machines, and I have had very much fun in the past playing with a small consumer drone which I received as a gift some years ago. This was one of the reasons that I decided on this project, it is something that I have always wanted to know more about but also something I find fun.

*Figure 1: taken from the TED website*

About a year ago, I watched two TED lectures by the autonomous systems pioneer Raffaello D'Andrea: Talk 2, Talk 1, where he demonstrates the current

capability of quadcopters and their athleticism. These two videos inspired me to start this project, so that I could learn more about this subject and try to understand in practical detail how these quads fly. He demonstrates and explains how engineers design algorithms to control flying machines, and shows how advanced modern control theory has become in areas such as teamwork and stability even after damage.

*Figure 2: also an Icon taken from the TED website*

## 1.3: Building

In this project, I had to choose an appropriate means of building the quadcopter, such that quality of the finished drone wasn't compromised but also making good use of the money and resources available. One of the options available was to buy an off-the-shelf quadcopter and fit is with my own control hardware to give me more freedom. I decided against this eventually because of the overhead cost that comes with consumer quadcopters from the control circuits and sensors – which I wouldn't use – and the limited space in the closed chassis of many large consumer drones.

After choosing this approach, I had to choose the form factor of the frame that I would buy. It had to be large enough to fit all of the sensors and microcontrollers for the drone but not so large that the peripheral components (like motors) wouldn't become too expensive. I decided upon the common 250 form factor (250 meaning 250mm across the diagonal), since there is a large selection of components available for it at reasonable cost, and it is large enough to fit all of the wires and control gear properly. There is also a large community of hobbyists who are familiar with this type of quadcopter and so there is a wealth of advice on choosing the components to suit my needs.

*Figure 3: A standard 250 quadcopter frame*

After choosing a frame, there are three more things to consider:

1. Motors
2. Speed controllers for those motors
3. Battery and power solutions

### 1.3.1: Choosing the Power Solution

All quadcopters nowadays use LiPo (Lithium Polymer) batteries. The reason for this is that they are fairly light and small, and have the ability to provide large amounts of current which is a requirement for the motors of quadcopter. Their smallness and lightness also diminishes the amount of energy spent per flight, which reduces the environmental cost of having to provide a large amount of energy for the quadcopter. The two most common types of battery for 250 quadcopters are 3S and 4S (3 and 4 cell). Each cell has a nominal voltage of 3.7V and so a 3S battery is 11.1V and a 4S battery is 14.8V. I chose a 3S LiPo because they are in general cheaper than 4S batteries and I am not in need of the higher motor torque given by a 4S battery.

For testing purposes, I wanted a large battery that could run for a long time without needing to be charged up. I used a 2200mAh battery as opposed to an 1800mAh battery (or less). This meant that

the drone had to gain more weight overall, but the larger battery size allows the drone's motors to function continuously for up to 20 minutes, which is far more ideal than the 5 or 10 minutes provided by smaller batteries.

Batteries also have a "C" rating, which is the quotient of the maximum current draw (A) with the capacity (in amp-hours). The higher this is, the more current the battery can supply and the lower the internal resistance of each cell. To charge the battery, I use a charger to send a constant overall current into it (2.2A for a 2200mAh battery) while monitoring the level of each individual cell and changing the current to each depending on its current charge level. This process is called "balance charging" and I use a charger to do this automatically.

### 1.3.2: The Motors

After some research, I found that there several numbers that describe a brushless multicopter motor's properties:

- The "kv" rating
- Four numbers that describe the size of the motor
- The coil and permanent magnet configuration

Every brushless motor has a "stator" and a "rotor". The stator contains several electromagnets, with their windings' axis pointing radially outwards, which are powered one after another to allow the motor to spin. The spinning part of the motor contains curved rare-earth magnets whose strong magnetic field causes the rotor to lock into a certain position when the coils are activated properly. There are three wires attached to the motor, each one being connected to sets of electromagnets the exact configuration depends upon the variety of motor. Brushless motors can either be "inrunners" or "outrunners". An inrunner motor has the stator outside of the rotor, with the shaft in the middle rotating with its permanent magnets. An outrunner is the opposite. The rotor is a shell which encases the stator and is driven from the inside. The outer part of the motor rotates at high speed. It is more practical to attach propellers to this design and so nearly all quadcopter motors are outrunners.

The "kv" rating describes how many revolutions per volt the motor can manage at maximum speed. The higher the voltage applied across the coils, the higher the current and hence a stronger magnetic field rises in the coils, making the rotor "snap" into position faster. For example, a 1000Kv motor driven at 12 volts has a maximum of 12,000RPM.

*Figure 4: The motor I eventually chose*

The first pair of four numbers the describe the motor's size state the inner diameter of the rotor in millimetres, and the second pair is the stator's height, also millimetres.

Finally, motors have two numbers associated with them that describe the motor's magnetic configuration. For example, a motor might by "12N14P", meaning that is has 12 coils on its stator and 14 permanent magnets.

I chose a fairly standard 2205 size motor, which runs at 2300kv and has a 12N14P magnetic configuration. I chose the size since it provides plenty of torque without being massive and also a fairly high "kv" for the quadcopter to be capable of high performance.

### 1.3.3: Speed Controllers

When turned by an external source, such as a hand or another motor connected to it, a brushless motor produces a three-phase AC on its output. This is caused by the changing flux through the electromagnets, creating a sinusoidal voltage sometimes called “back EMF” (interestingly, this voltage can be applied across a battery to drain the magnetic field energy from the coils and break the motor, the principle behind regenerative breaking used in modern electric cars). Three phase AC is just three sinusoidal voltages, each offset by 120 degrees:

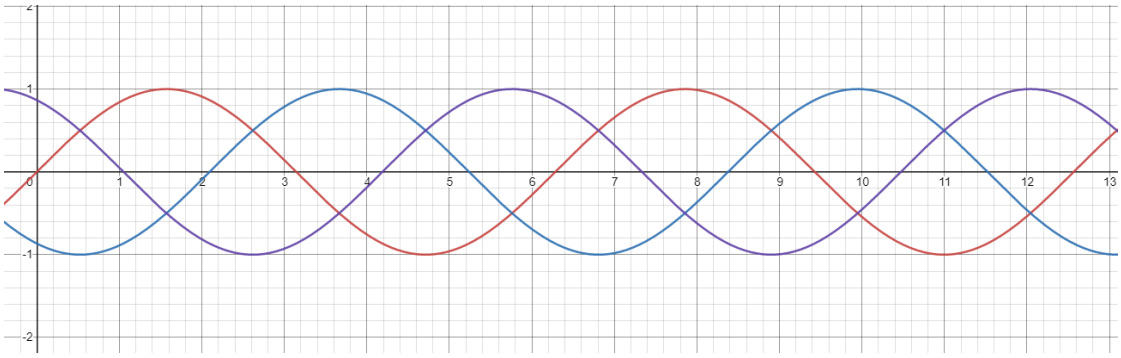



*Figure 5: Three-phase AC plotted on a graph using Desmos graphing calculator.*

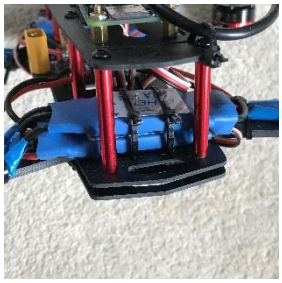


*Figure 6: A pair of speed controllers*

This back EMF persists even if you are driving the motor with a power source connected directly to it. This can then be used to determine the position of the motor in its rotation. This information can then be used to power the correct set of electromagnets to allow the motor to rotate smoothly. This is the purpose of an electronic speed controller (ESC for short). These voltages cannot just be applied by a microcontroller, since these motors need large currents to run, directly from the battery. Therefore, ESCs have two input wires for the battery’s DC, and three output wires that connect directly to the motor. They also have ground, 5V and signal wires which are broken out onto a female header which is safe to connect to a microcontroller. The 5V can also be used to power the control circuits, which is a very useful feature.

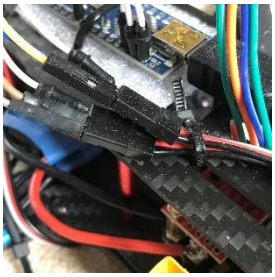


*Figure 7: The three header pins of the ESC*

I decided to use an ESC with a 20A current rating, since the lower 3S voltage of the battery cannot drive more than a total current of 80A or so, meaning that I never go above this limit. Also, the 20A ESC has a 10 second burst current capacity of 25A, which is well above the limit set by the battery.

## 1.4: Programming and Controlling

The other main part of the project is the control logic which actuates the propellers in an appropriate way so that the drone responds to commands properly and is able to maintain stable flight. There are commercially available control boards which handle all of the sensor data input and convert it into propeller commands. However, my plan for the project was to design all of the control software myself from scratch using programmable microcontrollers and sensors.

*Figure 8: A Raspberry Pi zero W measures only 30x65mm, image from the product listing on their page.*

To do this, I chose to use a Raspberry Pi Zero W board, along with an Arduino Nano, to actuate the propellers. Initially, I didn’t realise that the raspberry pi was not able to generate 4 independent signals for the

propellers – it is only able to generate two. I found this in testing and so I had so bought the Arduino microcontroller which has six channels to drive the propellers. I chose the raspberry pi specifically because they are very small and light, while having the hardware capabilities to run control logic. They also have a large array of interfacing options, mainly available through the GPIO (General Purpose Input/Output) pins, such as UART (Universal Asynchronous Receiver-Transmitter), $I^2C$ (Inter-Integrated circuit), and SPI (Serial Peripheral Interface) as well as a host of other pins that I can manually control as I wish.

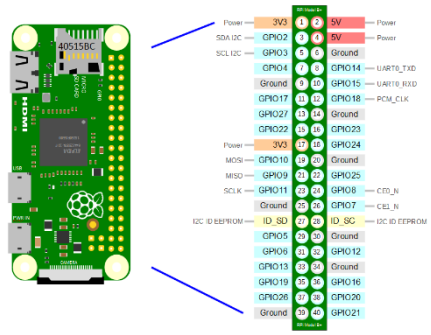



*Figure 9: The GPIO pinout diagram for the pi zero W*

I also have experience with Raspberry Pi boards, since I have owned one for over seven years and I have had to set up and install the operating system on them many times, as well as writing code using them, meaning that the setup and installation of the operating system was a familiar process. I chose the "W" variant of the Pi Zero since it has on-board Wi-Fi and Bluetooth capabilities allowing me to wirelessly control the quadcopter using either of these. The exact hardware specifications are listed below, from the official Raspberry Pi website: [https://www.raspberrypi.org/pi-zero-w/](https://www.raspberrypi.org/pi-zero-w/):

- 802.11 b/g/n wireless LAN
- Bluetooth 4.1
- Bluetooth Low Energy (BLE)

Like the Pi Zero, it also has:

- 1GHz, single-core CPU
- 512MB RAM
- Mini HDMI and USB On-The-Go ports
- Micro USB power
- HAT-compatible 40-pin header
- Composite video and reset headers
- CSI camera connector

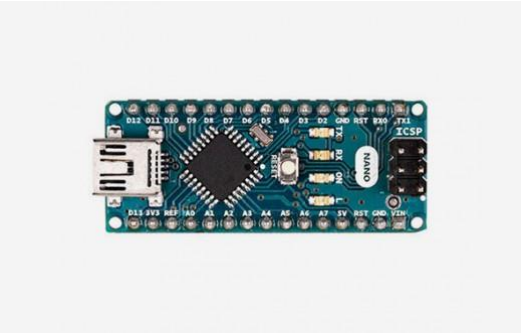


*Figure 10: Arduino nano, taken from https://store.arduino.cc/arduino-nano*

I have had no previous experience with the other microcontroller board which I used for this project, the Arduino Nano, which took me a few days' research to get started with, but they are much simpler than a raspberry pi, since they don't run an operating system. I used this to generate the signals to send to the speed controllers for the motors, which communicates with the pi via USB. It uses a variant of the C++ programming language which is compiled onto the microcontroller by a custom-built development environment on a laptop or PC.

As well as control boards, I needed sensors to sense things like altitude, attitude and acceleration in three-dimensional space. To measure height from the ground, I used an Elegoo HC-SR04 ultrasonic distance sensor, since they are inexpensive and provide a reasonable level of reliability and accuracy for indoor testing. Also, I chose the SparkFun breakout board of the MPU-9250 three axis accelerometer and gyroscope (and magnetometer) for acceleration and angular velocity data. I interfaced with the ultrasound sensor using the Raspberry Pi's GPIO pins, controlled manually. This sensor needs a short trigger pulse to activate it and it then sends back an echo pulse on the echo pin, telling the user how long the sound took to pass from it out to the surface (such as a wall) and back. The MPU-9250 is an $I^2C$ board, and so I used the two $I^2C$ pins on the raspberry pi to get data from it, as well as 3.3V and ground for a power supply. I will go into more detail about these sensors later in the report.

*Figure 12: The Elegoo HC-SR04 sensor, from their website*

*Figure 11:The Sparkfun breakout for the mpu-9250 sensor package*

## 1.5: Conclusion

In this section I have presented and discussed all of the hardware that I used in the project. In the future, I may choose to change some of the hardware or add more (something that comes to mind immediately is to add a barometer or thermometer to measure height and environmental conditions). To conclude, however, I thought that I should show a rough costing for the whole drone so the point that I made earlier about the pricing of the drone in comparison to consumer products stands:

- Frame: £25
- ESCs: £23
- Motors: £60
- Power Distribution: £7
- IMU and Ultrasound Sensor: £25
- Raspberry Pi: £10
- Battery: £23
- Charger: £50
- Propellers (16): £10
- Total: £233

# 2: The Project

## 2.1: Introduction

In this chapter I will follow the exact outline in 1.1 and present a complete, detailed start-to-finish description of the project's progress, and problems I had to overcome. The main decision making for the project was done at the research phase at the beginning. My findings surrounding this have been described in the first chapter. Now was time to begin with the project itself: programming and building to make these ideas into a finished product.

## 2.2: The Physics

Before I could do any programming or assembling, I had to decide on appropriate method of controlling the quadcopter. This task is impossible without first having a complete understanding of how the quadcopter and all of its parts interact to make it move around, and be able to maintain

stable flight. This is where I set pen to paper, and – after some research – wrote down the laws of physics that governed how the quadcopter moves. This analysis is by no means complete and I am still researching this. For my initial purposes of making the quadcopter hover, this is all that is necessary. However, there are details missed out, most importantly, the coupling and transformation between the coordinate system of the world and the coordinate system of the drone, which takes some linear algebra and matrix manipulations that I had not yet gained the mathematical capability for at the start of the project. This is something that I would like to do in the future with the project.

Although I am referencing no specific material, I would like to thank Walter Lewin for his extraordinary work in teaching physics as a professor at MIT. His 1999 8.01 lectures have proven an invaluable resource throughout this process and I have since gone on to watch all of 8.01, 8.02 and some of 8.03, from which I have gained much insight.

The drone has four motors, each one provides lift depending upon its speed. This force can be expressed by the equation:

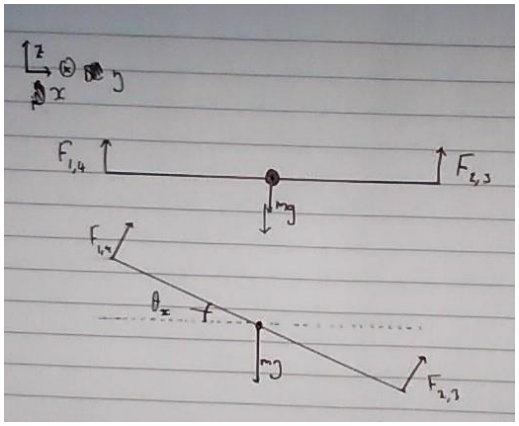
$$F_{ti} = C_L \omega_i^2$$

For a single propeller. C_l is a lift coefficient, which is dependent on things like propeller geometry and surface area, as well as air density. Omega represents the angular velocity of the propeller in radians / second. “i” is an index for the propeller, ranging from 1 to 4, starting from the front left propeller and going clockwise, ending on the back left. The total thrust from all four propellers can thus be expressed as follows:

$$F_{ttot} = \sum_{i=1}^{4} C_L \, \omega_i^2$$

In my calculations, I did not take the yaw angle of the quadcopter into account, since it cannot be measured directly by the sensors I have available, and is not something I wish to control. Pitch and roll, however, are. (Having said this, I am planning to be able to control the angular velocity of the drone on that axis).

If we rotate the drone around one axis like this:




We can resolve the forces in all directions. Clearly, $F_x = 0$ , since the quadcopter’s propellers’ force does not have a component in that direction, since $\theta_y = 0$. Now, we must decompose the force from the propellers into both the y and z direction as:

$$F_y = \sin(\theta_x)\, F_{ttot}$$

$$F_z = \cos(\theta_x)\, F_{ttot} - mg$$

This is fine, if we only have one axis which the drone can rotate around. In reality, we have two. Now, I had to think exactly what this meant for the forces in each direction of the world’s coordinate system. Surprisingly enough, a pure rotation about the y-axis does not affect the force in the y direction. This holds for the x-axis as well, so we have in general for those axes:

$$F_y = \sin(\theta_x)\, F_{ttot}$$

$$F_x = \sin(\theta_y)\, F_{ttot}$$

It is a little different for the z-axis. Since the force in the z-axis depends upon both angles, there are now two terms in front of the total thrust force term:

$$F_z = \cos(\theta_x)\cos(\theta_y)\, F_{ttot} - mg$$

All of the angles here are with respect to the world coordinate system, and so any values of acceleration that the quadcopter reads from its sensors will not be these values, but these values transformed into its coordinate system.

Now, I turn to the behaviour of the pitch, yaw and roll of the quadcopter over time, i.e., the calculation of the torques present and acting upon the quadcopter. For this section, we are firmly rooted in the body coordinate system of the drone. I gave very little thought to this at the start of the project, and so made the error of basing my calculations for the forces on the world coordinate system and the calculations for the torques on the body coordinate system of the drone (since it is easier). This is, quite frankly, a fairly grievous error, and I do aim to rectify this in the future, using the proper transformations using linear algebra.

About the x (roll) direction, the torque's is given by:

$$\tau_x = R(F_1 + F_4 - F_2 - F_3)$$

Where R is the distance of the propellers from the centre of the drone in the y direction. Similarly, the torque about the y (pitch) axis if given by:

$$\tau_y = R(F_3 + F_4 - F_1 - F_2)$$

In the z (yaw) direction, things become more interesting. Unlike roll and pitch, for yaw the thrust of the propellers has no direct contribution to the torque about the z axis. However, I knew from experience that drones have a way to control yaw. Typically, a quadcopter has two propellers spinning clockwise, and two spinning anti-clockwise. These are configured so that the propellers spinning in the same direction are diagonally opposite. This is because the clockwise-spinning propellers "want" to spin the quadcopter anti-clockwise and the anticlockwise-spinning propellers the opposite. After thinking about this for a time, I realised that this is because the drag clockwise-spinning propellers experience, by Newton's third law, exerts a torque on the quadcopter which is greater when that drag is acting on the portion of the propeller that is furthest from the centre of the drone. When the propellers exert a torque in this way, it can be expressed as:

$$\tau_{zi} = C_{mi}\omega_i^2$$

Where $C_{mi}$ is a coefficient depending on (importantly) drone radius, air density, propeller geometry etc. where the 'm' stands for "moment". $i$ is, again, an index from 1-4 for each propeller. Similarly to lift, the total torque acting on the drone about the z axis is simply:

$$\tau_{ztot} = \sum_{i=1}^{4} C_{mi}\, \omega_i^2$$

Now all of the torques and forces for the quadcopter are set out, I had to apply Newton's second law to them to extract the accelerations of the drone from them. In general, the law can be expressed as:

$$\boldsymbol{F} = \frac{d\boldsymbol{p}}{dt}$$

And:

$$\boldsymbol{\tau} = \frac{d\boldsymbol{L}}{dt}$$

Where bold-faced symbols represent vectors. Since the momentum of an object is defined as $\boldsymbol{p} = m\boldsymbol{v}$, and angular momentum $\boldsymbol{L} = \boldsymbol{I\omega}$, a product rule expansion for both of these yields:

$$\boldsymbol{F} = \boldsymbol{v}\frac{dm}{dt} + m\frac{d\boldsymbol{v}}{dt}$$

$$\boldsymbol{\tau} = \boldsymbol{\omega}\frac{d\boldsymbol{I}}{dt} + \boldsymbol{I}\frac{d\boldsymbol{\omega}}{dt}$$

Since the time derivatives of mass and mass moment of inertia in these are both 0, these simplify to the very familiar:

$$\boldsymbol{F} = m\boldsymbol{a}$$

$$\boldsymbol{\tau} = \boldsymbol{I\alpha}$$

Since we have the force and torque for each axis, we can now obtain the accelerations if we know the mass and mass moments of inertia of the drone. These were determined once I had built the drone completely. The next stage after I had done this was to use the python scripting language to write these physical laws into code, and to determine the position and angle of the drone at each point in time, given the velocities of the propellers.

## A Comment on Matrices

As I mentioned in the previous section, the way I handled the angles and orientation of the drone was far from ideal. This is because I derived all of the forces on the drone and handled them in the world's coordinate system i.e., a coordinate system that stays fixed with respect to the earth, and the drone moves about it, and the angles (angular accelerations) only with respect to the drone's coordinate system. Because of the length of time that this project has been in progress for, my understanding of this has increased up to a point where I am able to use linear algebra to properly transform the force and angular acceleration vectors between these two spaces.

In two dimensions, a rotation about the origin by an angle θ clockwise can be written as:

$$\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

This is intuitively pleasing, since I knew from intuition that any rotation in two dimensions doesn't "squish" or "stretch" the space at all, so the area of the unit square enclosed by the two unit vectors (the determinant of the matrix) must be 1, which it is. This matrix generalises nicely to three dimensions, since the $\hat{\boldsymbol{z}}$ unit vector stays where it is, and this matrix becomes:

$$\begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Since this matrix represents rotation about the z axis, for the rest of this section I will denote it as $\boldsymbol{Z}(\psi)$ for the rest of the section. For the x and y axes, the situation is very similar, except that now,

we are rotating a different set of unit vectors. For rotation about the x-axis, the x-axis stays in place so:

$$\boldsymbol{X}(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & \sin\theta \\ 0 & -\sin\theta & \cos\theta \end{bmatrix}$$

And similarly for the y-axis:

$$\boldsymbol{Y}(\varphi) = \begin{bmatrix} \cos\varphi & 0 & -\sin\varphi \\ 0 & 1 & 0 \\ \sin\varphi & 0 & \cos\varphi \end{bmatrix}$$

I use different symbols for the above angles to make the mathematical distinction between the potentially different values that are being plugged into these matrices. Now we have this, the composition of these three matrices must give the full rotation:

$$\boldsymbol{R}(\theta, \varphi, \psi) = \boldsymbol{Z}(\psi)\boldsymbol{Y}(\varphi)\boldsymbol{X}(\theta)$$

If we have a vector expressed in terms of the world's coordinates, transforming it using this matrix would give us the new vector in the body coordinate system. To do the opposite, i.e., transforming the vector back, we must use the inverse of this matrix, $\boldsymbol{R}^{-1}(\theta, \varphi, \psi)$. Using this result we now have the tools to convert the force, and therefore the acceleration exerted by the propellers of the drone, F_ttot, in the world coordinate system:

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \frac{1}{m}\boldsymbol{R}^{-1}(\theta, \varphi, \psi) \begin{bmatrix} 0 \\ 0 \\ F_{ttot} \end{bmatrix}$$

These three angles are called Euler angles and they have the specific property that in order to get the desired orientation of the resulting vector from the rotation matrix they have to be consistently applied with x-axis first, then y, then z. And the opposite applies for the reverse transformation. The conversion to Euler angles from the data given by the IMU is a difficult task, and since I never used them in the original implementation of the sensor code they are beyond the scope of this report. However, I am doing ongoing learning and research into this, and it is something that will most certainly implement in the future.

Although the explanations are mine, I checked my derivations of the governing matrices against the article at: https://towardsdatascience.com/demystifying-drone-dynamics-ee98b1ba882f

## 2.3: Implementing the Physics – Python Project

### 2.3.1: Introduction

The goal of understanding the physics of the drone was eventually to translate this into a working computer model of the drone, so that I could conduct experiments and tuning for the control system on a virtual representation, without the inherent risk of live testing. Another reason for this was simply that I did not have access to the hardware necessary to do this at that stage of the project.

At the end of the last section (discounting matrices), what we essentially have is 6 coupled differential equations. These equations are very difficult therefore to solve analytically, but numerically there are no problems. This is the route I decided to take. Therefore, this section is dedicated to my numerical solution of those equations and the steps and problems upon that journey. Up until this point all of the report has been theory: it is time to put these ideas in to practice(!) I am using the python scripting language here because I am very familiar with it, and it is a

fairly high-level language that is therefore good for algorithm development, since it is easy to read and write. Unfortunately, it is not as efficient as a language like C++, but it serves my purpose well and – for the calculations that I need to do – is perfectly sufficient. Here are the equations as they stand at the end of the last section:

$$F_{ttot} = \sum_{i=1}^{4} C_L\, \omega_i^2$$

$$\ddot{z} = \cos(\theta_x)\cos(\theta_y)\frac{F_{ttot}}{m} - g$$

$$\ddot{y} = \sin(\theta_x)\frac{F_{ttot}}{m}$$

$$\ddot{x} = \sin(\theta_y)\frac{F_{ttot}}{m}$$

$$\ddot{\theta}'_x = \frac{R}{I_x}(F_1 + F_4 - F_2 - F_3)$$

$$\ddot{\theta}'_y = \frac{R}{I_y}(F_3 + F_4 - F_1 - F_2)$$

$$\ddot{\theta}'_z = \frac{1}{I_z}\sum_{i=1}^{4} C_{mi}\, \omega_i^2$$

The primes here indicate that the quantity is changing with respect to the body coordinate system of the drone, as opposed to the more general world coordinate system. Notice that I have isolated all of the acceleration terms on the left-hand side.

This is all very well, but these equations only allow us to directly calculate the accelerations of the drone in each axis (angular and linear). This means that in order to calculate the complete state of the drone, we must extrapolate the velocity and position data from the acceleration. Usually, to obtain velocity from some acceleration function, $a(t)$, we have to perform the integral:

$$v(T) = \int_0^T a(t)dt + v_0$$

Similarly, to obtain distance from velocity:

$$x(T) = \int_0^T v(t)dt + x_0$$

However, there are two issues with this approach that mean that it cannot be implemented in code:

a) The acceleration function can be a completely arbitrary function of time, and so this integral is unevaluable analytically in most cases;
b) Computers cannot (easily) evaluate integrals analytically anyways.

We need another way. Fortunately, there is a reasonably simple and tested way to do this. Fundamentally, an integral is a limit of a sum of small rectangles' area whose height are the function's height at that point and their width is a small increment to the function. When and

integral is expressed directly in this way, this is called a Riemann sum. We cannot take the limit in this case with a computer, but the velocity and distance can be approximated to reasonable accuracy by taking the small change in input to the function to be finitely large. Executing this sum will give more accurate results for smaller values for the increment in the input variable (in this case, time, $t$).

This results in some nice graphical intuition, since an integral can be thought of as the area under a curve, and so a Riemann sum is just what happens when you sum over finite slithers of the graph's area, which under or over-estimates some area of the graph, but it is a good enough approximation to do modelling with, provided that the timestep, $\Delta t$, is small enough.

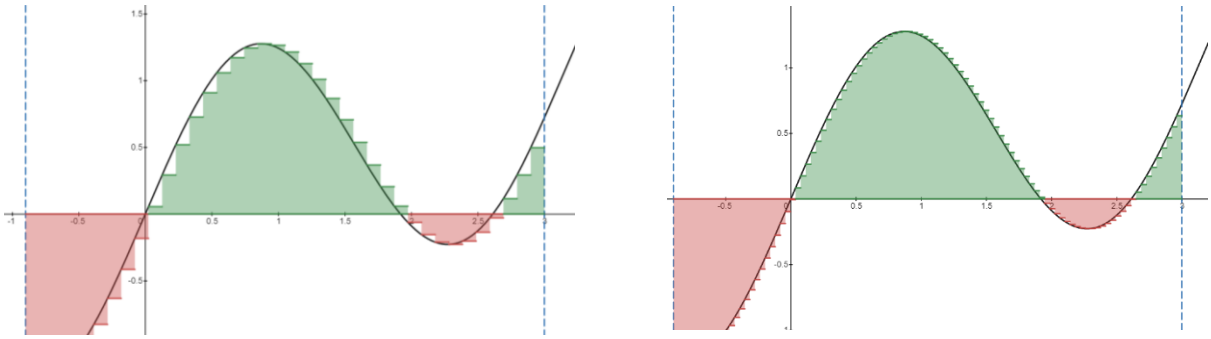


*Figure 13: A coarse Riemann sum, from a Desmos applet, left, and a finer version on the right, demonstrating the increased precision gained by using thinner slices.*

Now that this is all set out, I had to begin to write a computer program to implement this modelling. This was developed over a series of about two months, and when I had finished, I was able to test my ideas for a control system with it.

### 2.3.2: Starting on The Code

The language that I used for this program was the python scripting language. It was the language that I was most accustomed to at the time, since I have had many years using it to write small programs and games for fun.

I decided to separate my code into 3 files for ease of development. One called `main.py`, which handles all of the runtime calculations and calls to functions, `DroneSystem.py`, which contains the implementation of the drone class, `constants.py`, which holds any necessary physical data about the drone (such as lift coefficients) and `droneSystemLogic.py`, which contains any auxiliary functions that are useful calculations that the Drone System may need to call upon to do its job.

It is also worth noting that this is not the only numerical method for integration that exists, for example, it is as possible to assume that the slices of the graph are trapezoids. This provides more accuracy, and is something that I may consider implementing in the future. However, my results with this other simpler method were satisfactory, and so I did not feel the need to rewrite experimental code that would only be improved in a limited way by the other method.

Here is the entirety of the constants file:

```
import math

# ENVIRONMENT
PRECISION = 5
TIME_STEP = 0.02
```

```
DENSITY = 1.225
GRAVITY = 9.81
FLOORZ = 0
PROP_STEP = 1.27

# LIFT AFFECTING DATA
PROPELLER_AREA = 0.001
PROPELLER_RADIUS = 0.05
LIFT_COEFFICIENT = 0.7

# DRONE FRAME DATA
DRONE_RADIUS = 0.125
PROPELLER_DISTANCE = math.sqrt(2*(DRONE_RADIUS**2))
TORQUE_RADIUS = PROPELLER_DISTANCE/2
DRONE_MASS = 0.581

# ROTATION AFFECTING DATA
MOMENT_COEFFICIENT = 0.01
MOMENT_OF_INERTIA_Z = 2.782
MOMENT_OF_INERTIA_Y = 1.35
MOMENT_OF_INERTIA_X = 1.47
```

Some of the things in this file need further explanation:

- Precision: set to 5 here. This is the number of decimal places that everything in the rest of the program is rounded to. This is mainly since I prefer to work with rounded values for printouts (it's clearer and easier to read), and I prefer to have 0.1 staring back at me rather than the 0.09999999999999999999 that would be caused by computational rounding error (0.1 is a recurring decimal in binary and so would need an infinite amount of memory to express exactly).
- The time step is the width of the slithers of area to add up for the numerical integral.
- The drone radius is measured diagonally, so the radius at which the torque is applied is calculated using Pythagoras' theorem.
- Everything is in SI units here.

Now that this was done, the next step was to begin work upon the various utility functions in the droneSystemLogic module. This layout allowed me not to worry about the various computations that naturally go along with all of the acceleration calculations (as laid out the in the equations) within the class that implements the various properties of the drone and marches them forward in time. These functions are:

- Lift calculations for a single propeller, given its angular velocity;
- Lift calculations (hence) for the whole drone, given an array of four rotational velocities.
- The same above but for moments;
- Some simple functions that calculate the accelerations of the drone given some force.

For completeness, I will list the code for all of these below.

Lift for a single propeller:

```
def calculateLift(inputThrottle):
    liftOutput = 0.0009 * inputThrottle ** 2 + 0.0059 * inputThrottle
    return round(liftOutput, PRECISION)
```

This is strange, since the expression used to calculate the lift does not take the $C_L\omega_i^2$ discussed in the previous section. This is actually not the original form of the function, which did take the form above, since these numbers were collected after a physical test and a best fit polynomial approximation of the resulting data (to be discussed in detail in part 9 of this chapter).

Lift for the whole drone ($F_{ttot}$):

```
def calculateThrust(rotationalVelocityList):
    thrustTotal = 0
    for rotationalVelocity in rotationalVelocityList:
        thrustTotal += calculateLift(rotationalVelocity)

    return thrustTotal
```

This takes a four element array of all of the rotational velocities, converts them into lift and sums them.

The calculations for yaw moment are very similar:

```
def calculateMoment(rotationalVelocity):
    momentOutput =
MOMENT_COEFFICIENT*DRONE_RADIUS*rotationalVelocity**2
    if rotationalVelocity < 0:
        momentOutput = -momentOutput

    return round(momentOutput, PRECISION)


def calculateYawMoment(rotationalVelocityList):
    yawTotal = 0
    for rotationalVelocity in rotationalVelocityList:
        yawTotal += rotationalVelocity

    yawMoment = calculateMoment(yawTotal)
```

There are also some functions that take a moment or force and convert it into (angular) acceleration.

```
def calculateAcceleration(force):
    acceleration = force/DRONE_MASS
    return round(acceleration, PRECISION)


# these three function are independent since the moments of inertia
# are (constant) global variables, so it is not necessary to pass them
# into a single function
def calculateRotationalAccelerationZ(moment):
    rotationalAcceleration = moment/MOMENT_OF_INERTIA_Z
    return round(rotationalAcceleration, PRECISION)


def calculateRotationalAccelerationY(moment):
    rotationalAcceleration = moment/MOMENT_OF_INERTIA_Y
    return round(rotationalAcceleration, PRECISION)


def calculateRotationalAccelerationX(moment):
    rotationalAcceleration = moment/MOMENT_OF_INERTIA_X
    return round(rotationalAcceleration, PRECISION)
```

There is one more algorithm in this file which is a little bit more than a simple calculation. This is designed to convert (thrust, roll, pitch, yaw) into four propeller speeds. This is not used directly in the drone class, but is very useful to me later on and is used in the code on-board the drone which converts controller output into propeller signals. The idea here is that the thrust argument is added to all four propellers, the roll added to the left pair and subtracted from the right, the pitch added to the back pair and subtracted from the front pair, and yaw added to the two clockwise-spinning propellers and subtracted from the anticlockwise spinning propellers (anticlockwise rotation viewed from above is considered positive). Here it is important that the values entered don't lead to a negative velocity for one of the propellers, so I wrote the function such that it would throw an error if this happened. Here is the code:

```
def motorMixer(thrust, roll, pitch, yaw):
    motorVelocityList = []
    assert abs(roll+pitch+yaw) <= thrust, "The values you have entered
will cause the system negative prop velocities"

    motorVelocityList.append(((thrust / 4 + roll / 4 - pitch / 4 + yaw
/ 4) // PROP_STEP) * PROP_STEP)
    motorVelocityList.append((-(thrust / 4 - roll / 4 - pitch / 4 - yaw
/ 4) // PROP_STEP) * PROP_STEP)
    motorVelocityList.append(((thrust / 4 - roll / 4 + pitch / 4 + yaw
/ 4) // PROP_STEP) * PROP_STEP)
    motorVelocityList.append((-(thrust / 4 + roll / 4 + pitch / 4 - yaw
/ 4) // PROP_STEP) * PROP_STEP)
    return motorVelocityList
```

Normal order of operations applies. The "PROP_STEP" variable was added later, and its function is to only allow certain increments of speed of the propellers (as is possible on the physical quadcopter).

This concludes the section on this part of the code.

### 2.3.3: The Drone Class

This section is dedicated to my development of the actual drone class. This is really the hard of this whole program, and it is where all of the numerical integration and calculation actually happens. I presented the "droneSystemLogic" last section because it is used extensively in this class and so makes it easier to read.

Classes are a feature of object-oriented programming that I used for the Drone system. There are many schools of thought about the different types of programming, but I used a class here really for the purpose of being able to encapsulate the drone's data and functions so all I had to do in the main program was tell the drone what to do and it would do it itself, without me having to get bogged-down in the implementation details.

To start work on the class, I first had to define all of the variables and data that the drone handles. This is usually done in a "constructor", a function which is run immediately as soon as an instance of the drone is created in memory. Here is the code (just lots of initialisations):

```
class DroneSystem:
    def __init__(self):
        self.propVelocities = [0, 0, 0, 0]

        self.accelerationVector = [0, 0, 0]
        self.velocityVector = [0, 0, 0]
        self.positionVector = [0, 0, FLOORZ]
```

```
        self.accelerationAngles = [0, 0, 0]
        self.velocityAngles = [0, 0, 0]
        self.positionAngles = [0, 0, 0]

        self.weight = GRAVITY * DRONE_MASS
        self.thrust = 0
```

“self” is used whenever the variable (or method) after the ‘.’ is a member of the class. If self is not used, the python assumes that it is a variable that only exists within the function, and is destroyed once the function has finished.

The next functions in the class are the functions that determine the XYZ linear and angular accelerations using the equations that I derived at the beginning of section 2.2. They are a direct translation of the equations into code so I will not waste space by writing each and every one of them here. I will list two of the six: calculation of X angular acceleration and Z linear acceleration.

Z linear acceleration:

```
def calculateZAcceleration(self):
    zAcceleration = self.thrust * math.cos(self.positionAngles[0]) * math.cos(self.positionAngles[1])
    if zAcceleration - self.weight < 0 and self.positionVector[2] == FLOORZ:
        self.accelerationVector[2] = 0

    else:
        self.accelerationVector[2] = calculateAcceleration(zAcceleration - self.weight)
```

X angular acceleration:

```
def calculateXAlpha(self):
    propMoment = TORQUE_RADIUS * ((calculateLift(self.propVelocities[0]) +
                                   calculateLift(self.propVelocities[3])) -
                                  (calculateLift(self.propVelocities[2]) +
                                   calculateLift(self.propVelocities[1])))
    xMoment = propMoment
    self.accelerationAngles[0] = calculateRotationalAccelerationX(xMoment)
```

After this stage was complete, I now had to use these computed accelerations and integrate them in order to gain velocity and position information. There are two functions in the drone class for this purpose. The first updates the vectors and does the integration, and the second updates the entire system, as well as calculating the accelerations. The physical time between each update is TIME_STEP, and each time step the velocity is incremented by the acceleration multiplied by this timestep, and the distance is the velocity multiplied by the timestep. This is done for all of the axes.

This is the numerical integration technique that I discussed at the beginning of the section. The code for both is as follows:

```python
def updateVectors(self):
    for i in range(len(self.positionVector)):
        self.velocityVector[i] += self.accelerationVector[i] * TIME_STEP
        self.positionVector[i] += self.velocityVector[i] * TIME_STEP
        self.velocityAngles[i] += self.accelerationAngles[i] * TIME_STEP
        self.positionAngles[i] += self.velocityAngles[i] * TIME_STEP
        self.velocityVector[i] = round(self.velocityVector[i], PRECISION)
        self.positionVector[i] = round(self.positionVector[i], PRECISION)
        self.velocityAngles[i] = round(self.velocityAngles[i], PRECISION)
        self.positionAngles[i] = round(self.positionAngles[i], PRECISION)

    if self.positionVector[2] <= FLOORZ:
        self.positionVector[2] = FLOORZ
        for i in range(len(self.positionVector)):
            self.velocityVector[i] = 0
            self.accelerationVector[i] = 0
            self.positionAngles[i] = 0
            self.velocityAngles[i] = 0
            self.accelerationAngles[i] = 0
```

This is the function that does the integration and updates the vectors. The last "if" statement is a check to see if the drone has hit the ground. If it has, then it resets all of the drone's data to the default values except the x and y position and the propeller velocities.

```python
def updateSystem(self):
    self.thrust = calculateThrust(self.propVelocities)
    self.calculateXAlpha()
    self.calculateYAlpha()
    self.calculateZAlpha()
    self.calculateXAcceleration()
    self.calculateYAcceleration()
    self.calculateZAcceleration()
    self.updateVectors()
```

This function is the function that the person using the drone class has to call to update everything about the drone, as well as calculating the accelerations. This is very useful, as it only takes one line of code to update the whole drone, from the outside.

### 2.3.4: Conclusion

This project took up much of early 2020 lockdown. After it was in a working state, I was able to begin work coupling a control system to it, so that I might have a complete working mathematical model of the drone to do experiments, tuning and tests upon, without the need for any hardware. The physical assembly of the drone comes later in this chapter, and I used many of my results from this program to give to the code that runs live on the drone.

The next section is the control theory and logic that I chose to use for the drone, and how I coupled it to this model of the drone. This is also the section where I will develop "PID.py", which I have not discussed in this section.

## 2.4: The Control System and Its Implementation

### 2.4.1: Introduction

This is where I actually started writing code that would be part of the drone's software. After all of the mathematical modelling, I did some research and decided on how I wanted to keep the drone in the air.

Quadcopters are inherently unstable. Without active feedback control, they will not stay flat and fall out of the sky. This can be caused by wind, or simply the fact a drone's centre of gravity is never exactly in its physical centre, causing the propellers' torque to spin it over and crash it.

The process to combat this effect is known as “feedback control” which aims to take data from sensors on the drone and finely tune the propeller speeds multiple times a second to keep the drone in a specified position. This section is about how I chose to do this and also how I chose hardware (and wrote software) for its implementation.

### 2.4.2: Initial Ideas for the Control System

After doing some research and some thinking, I came up with two possible ways of controlling the drone:

a) Using a form of genetic algorithm that uses the NEAT (NeuroEvolution of Augmenting Topologies) algorithm;
b) Using a more traditional approach with the PID controller.

Option a was my first idea. This algorithm, essentially, uses a large population of neural networks to optimise a solution. A neural network is a structure that takes an array of inputs and converts them into (generally fewer) outputs, through a process of layers; each node in each layer holding a value. The idea is that each connection from node to node has a specific “weight” assigned to it, and this weight determines the “strength” of the connection and therefore how much a specific node’s value influences the value in the node it is connecting to. In this way, a network with the correct connection strengths can be given different sensor data on its inputs, and generate appropriate thrust, roll, pitch and yaw values to keep the drone aloft. For example, in this network there are three input neurons. Say that they had values [0.43, 0.87, 0.5]. The first neuron down in the “hidden” layer has three connections (one from each neuron in the input layer). Say that they had weights: [0.4, -1, 0.98]. The value of this neuron would become:

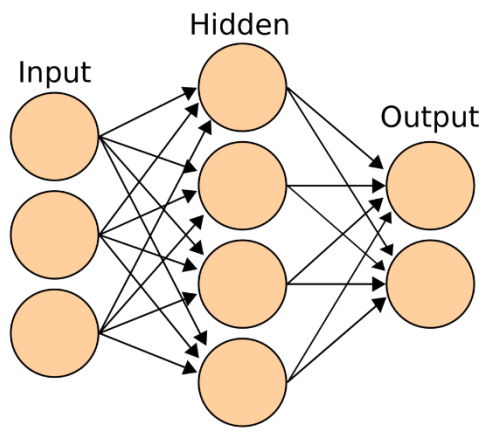



*Figure 14: A neural network (diagram taken from an article on sitepoint.com*

$$a = 0.43 * 0.4 + 0.87 * (-1) + 0.5 * 0.98 = -0.208$$

Since neuron’s values have to be between zero and one, this value would then be normalised. There are potentially thousands of these networks created in a program to provide a diverse population of solutions. The NEAT algorithm aims to find the best solutions and optimise them.

The benefits of this approach are really only computational speed (onboard the drone) and the fact that I had a working implementation of NEAT ready to go.

I will not go into only further detail on this approach, since I decided against using it. This is because once a solution has been trained, it is very difficult to tweak it. Also, it takes a lot of computer time to train a model, which is something that I would rather avoid.

Option b is more traditional. A PID controller is another way of controlling a system, and is more customisable.

### 2.4.3: The PID Controller

A PID controller is always used in a “feedback loop”. Its job is to take data from a sensor and compare it to a desired output, eventually creating output to tell the quadcopter’s propellers (often called actuators) exactly how fast they should spin. This process is called a feedback loop because when it is represented schematically, it looks like a closed loop:

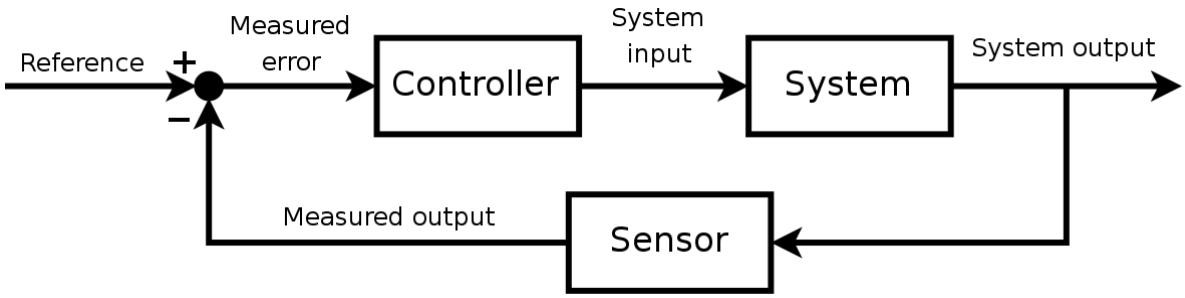



*Figure 15: A block diagram of a control loop from Wikipedia*

A PID controller can either be a physical device, or implemented in software. “PID” stands for “Proportional, Integral, Derivative” controller. I will develop the logic behind it in this section.

Say, for example, you are trying to lift up a weight to a specific height, or put a coat on a hanger. One way to approach this could be to look at the current position of the weight from the ground and calculate how far there is to go to the desired value. Then, set the speed of your hand equal to some proportionality constant – which you may choose - multiplied by this distance (also called error). In other words:

$$\dot{x} = k(h - x)$$

This differential equation is easily separable, so is fairly trivial to solve. This would result in your hand gradually rising, becoming slower and asymptotically approaching the desired height h:

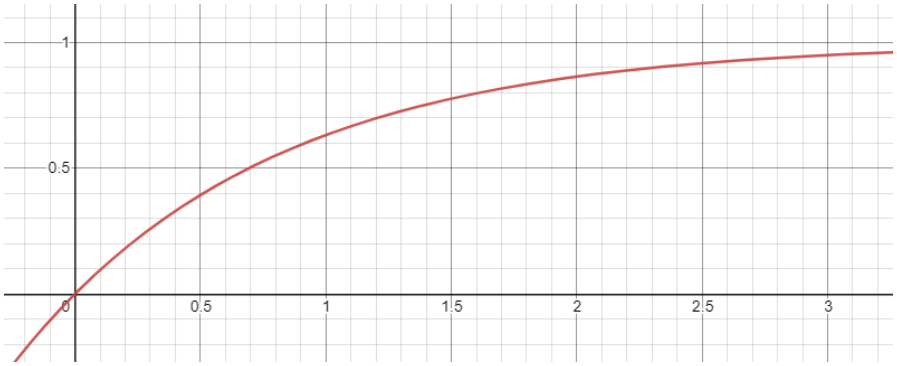



*Figure 16: The solution to the equation for k=1*

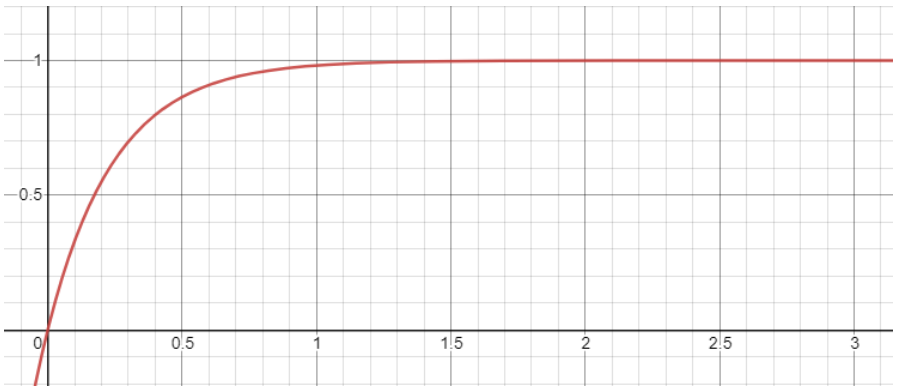



*Figure 17: Note that the curve approaches its maximum value faster for higher k (=4)*

For the drone, the velocity on the left-hand side of the differential equation would become propeller velocity, with the height being measured by a sensor.

This is sometimes all that we need for a controller. However, there is a rather large flaw that means that it is not possible to use this single philosophy with a quadcopter. The issue is that the output of the controller (the propeller velocity) is always 0 when the drone is at the desired height. This would result in the drone falling out of the sky if left at that point, which is certainly not suitable.

The solution to this is the 'I' or "integral" part of the controller. This aims to avoid the issue by taking the error and multiplying it by another weight each time the controller is updated. For example, if the quadcopter was two metres below the desired height, the controller would add k*2 to the output signal, and in general keep adding k*error until the quadcopter reached the desired height. This, unfortunately, is still not perfect.

The problem now is that the drone will tend to overshoot the desired value because the integrator, in effect, takes into account the past state of the quadcopter, and does not respond very fast to change. The solution to this problem is to add a third element to the controller, a 'D' or "derivative" element. This is the final part of a PID controller. This takes the rate of change of the error (its derivate with respect to time) and multiplies it by a third weight. This, in effect, takes into account the projected future state of the quadcopter and can be helpful to drive it to this state or slow it down as is reaches it so as not to overshoot.

To generate the total output of the controller, these three channels are summed together in order to create a total output signal.

The above has been almost completely qualitative, but it lays a good foundation for the philosophy of the basic underlying mechanism for control of the quadcopter.

### 2.4.4: Python Code for the PID controller

The python code for this idea is implemented, once again, in a class. This is because it is a construct that is able to abstract its functionality (generating output from sensor data) from its implementation. This means that I can use the general code for anything that I want to control, and I only have to write it once.

The code is as follows:

```
class PID:
    def __init__(self):
        self.p_weight = 0
        self.i_weight = 0
        self.d_weight = 0

        self.p_output = 0
        self.i_output = 0
        self.d_output = 0
        self.total_output = 0

        self.sensor_data = 0
        self.desired_data = 0
        self.error = 0
        self.old_error = 0

        self.floor = 0
        self.ceiling = 100

    def calculate_error(self):
        self.error = self.desired_data-self.sensor_data
```

```
    def calculate_p(self):
        self.p_output = self.p_weight*self.error

    def calculate_i(self):
        self.i_output += self.i_weight * self.error * TIME_STEP

    def calculate_d(self):
        self.d_output = self.d_weight*((self.error-
self.old_error)/TIME_STEP)

    def update(self, sensor_data):
        self.old_error = self.error
        self.sensor_data = sensor_data
        self.calculate_error()

        self.calculate_p()
        self.calculate_i()
        self.calculate_d()

        self.total_output = self.p_output+self.i_output+self.d_output
        if self.total_output >= self.ceiling:
            self.total_output = self.ceiling

        if self.total_output <= self.floor:
            self.total_output = self.floor

    def reset(self):
        self.p_output = 0
        self.i_output = 0
        self.d_output = 0

        self.sensor_data = 0
        self.desired_data = 0
        self.error = 0
        self.old_error = 0
```

Very similarly to the drone class, this class has some simple, small utility functions (such as calculate error) and a single, larger function which the user calls in order to generate the output. This "update" function calculates the error, then the individual channels, finally taking the sum and generating its output. Notice that this code caps the output of the controller in between two bounds. This is a rather dirty fix to the fact that the propellers cannot be driven faster than their maximum rpm, and so I have to stop the controller from telling them to do so. This issue is called actuator windup and the preferred solution is to not update or "turn off" the integrator channel when this happens.

I will implement this in the future (just after I finish writing this), but it is a good example of a problem which I had to fix during development of the software.

### 2.4.5: Applying the PID to the Quadcopter

Once I had written I PID, I had to decide how to couple it to the four propeller speeds on the quadcopter to successfully control it. There is a very natural seeming choice for this. On a standard drone controller, there are control joysticks for thrust, roll, pitch and yaw. This can be directly applied to the PID philosophy, each of these four parameters having a PID associated with it, each controlling a virtual "joystick".

In order for a PID to work properly, it must take data from a sensor, and compare it to a desired value, controlled by the user. I had to choose a place for each PID to get its data from that would make sense:

- Thrust: vertical distance from the ground (ultrasound sensor, altimeter (barometer))
- Roll: roll tilt angle (accelerometer)
- Pitch: pitch tilt angle (accelerometer)
- Yaw: yaw angular velocity (gyroscope)

This is the mapping that I chose, and the one that I think makes the most sense. The roll tilt angles can be directly determined from the accelerometer. This was something that I realised after some thought.

The accelerometer measures, in each axis, how much acceleration there is in each axis. Sitting level on the ground, both the x and y components of acceleration would be zero, since there is no gravity acting in those directions, however, the z axis reads $9.81m/s^{2,}$ since there is a perceived acceleration upwards constantly caused by the earth's gravity (this is really a relativistic idea, the fact that the curvature of spacetime around the earth results in a necessary acceleration upwards to maintain the same spatial coordinates, which is a fascinating topic in itself).

When the accelerometer is tilted, it measures an acceleration in one of its sideways axes, which can be normalised and taken the inverse sine of to determine the angle. The interesting thing is that this component is only affected by external forces on the drone (such as drag, which is negligible at low speed) and not the force from the propellers, since they are perpendicular to the sideways axis, and hence have no component of force in that direction.

This is the coupling of PID to quadcopter that I took forward into testing

### 2.4.6: Conclusion

This section has been a fairly qualitative tour of the control techniques that I chose for the quadcopter. The implementation in code is fairly simple. This is a marvellous thing to be able to achieve in any piece of code: simplicity and generality towards a solution of a seemingly complex task. This control system, although functional, is not completely perfect. There are some serious questions to ask about what happens when drag is not negligible, for example. Also, for the system I have essentially decided that thrust from the quadcopter only acts in the vertical spatial direction, which is often a very good approximation for small angles, since the cosine of an angle is approximately 1 for these angles. These imperfections will, unfortunately put limitations on the current control system, but they are limitations that will be addressed and removed as I learn more, and as the project goes on.

## 2.5: The Sensors

### 2.5.1: Introduction

This section may seem like something of a detour, since I have left hanging the idea of testing the control system for now. However, every control system must have reliable data to base its output from: this data is obtained from sensors, which must be managed in hardware as well as software.

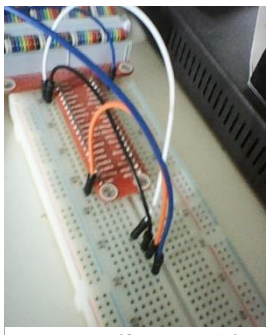


*Figure: My (fairly crude) sensor testing setup*

Although it may seem easy at first, the task of obtaining data from a sensor is far from trivial: it is something that I spent some months on. There are two main sensors that I will be getting data from for the physical hardware of the drone (when using the software model, sensor data can be obtained directly from

memory at the time of running the program): the HC-SR04 ultrasound distance sensor, and the MPU-9250 3-axis accelerometer. I explained my reasoning for choosing each of these at the end of section 1.4.

### 2.5.2: The HC-SR04 Sensor

This sensor is comparatively simple on its pinout. There are four lines, as in the figure, where wires connect to the sensor. The far left and far right are power input (5V) "Vcc" and ground (0V) "Gnd".

*Figure 18: The pins of the HC-SR04*

The other two pins are where most of the action takes place.

The first pin is "trig" – short for trigger, and the second is echo. In order to activate the sensor, the trigger pin is pulled up to 5V, then down to 0V in quick succession, creating a pulse on that pin. The sensor itself then sends a pulse of ultrasound from its transmitting speaker, which hits its target, and bounces off. Before the sound is returned, the sensor outputs 5V on the "echo" pin, which can be read by the device that it's connected to it. As soon as the sensor gets a return pulse to its receiving diaphragm, the pin is immediately pulled low.

This process allows a host device to determine the time that sound took to reach its destination, bounce and return. This is done by starting a software timer when "echo" is at 5V, and stopping it as soon as it detects 0V on the pin: the microcontroller simply keeps polling the sensor. Once this is known, it is simple to calculate how far the sensor is from its target:

$$distance = sound\ velocity \times \frac{time}{2}$$

The software that implements this process is a simple function that carries out the above algorithm.

```
def distance():
    gpio.output(PIN_TRIGGER, 1)
    time.sleep(0.00001)

    # deactivate
    gpio.output(PIN_TRIGGER, 0)

    pulse_start_time = 0
    pulse_end_time = 0

    # calculate times
    while gpio.input(PIN_ECHO) == 0:
        pulse_start_time = time.time()

    while gpio.input(PIN_ECHO) == 1:
        pulse_end_time = time.time()

    # find duration
    pulse_duration = pulse_end_time - pulse_start_time

    # calculate distance
    distance = round(pulse_duration * 17150, 3)

    return distance
```

### 2.5.3: The MPU-9250 Sensor

This sensor is far more difficult to operate than the HC-SR04.

As in the figure below, the first choice is deciding which set of pins to use for communication. Like the ultrasound sensor, there are two pins for power (in this case 3.3V) and ground.

Unlike the ultrasound sensor, this sensor uses either the SPI (serial peripheral interface) or the $I^2C$ (inter-integrated circuit interface). These are different in the sense that they base the transfer of data upon the transfer of individual bytes to and from the sensor. This has the advantage that it is extremely fast, and – given the complexity of the data – requiring of far fewer wires. Both of these interfaces were invented in the 1980s, and have since become de-facto standards.

*Figure 19: The MPU-9250*

I eventually chose $I^2C$ over SPI since it only requires two wires (as opposed to 4) and is generally more popular amongst the community because of its comparative ease of use.

$I^2C$ has two wires associated with it. A clock wire (often labelled SCL) and a data wire (SDA). The clock wire is a simple toggle signal between a logical 1 (in this case 3.3V) and a logic 0:

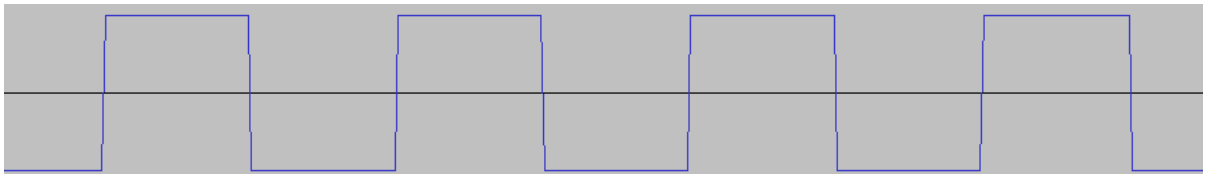


*Figure 20: The clock signal*

The other wire is a data wire. For example, eight bits of information transfer across this wire might look like:

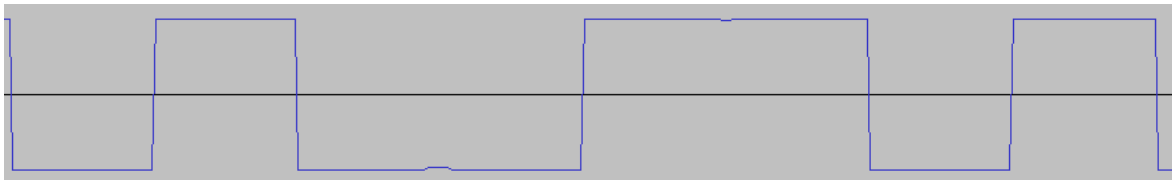


*Figure 21: Potential data*

This system is important since both devices must agree exactly how much time there is in between bits. If the clock speed is even microseconds different between devices, the data read will become completely garbled very quickly, since this system's clock runs at 100kHz in standard mode, leaving little margin for error. It is possible for both devices to agree upon a clock rate before data transfer, but this is inherently unreliable due to the fact that the devices must independently generate a clock signal internally, which is subject to minor variations in hardware that can possibly lead to data corruption.

I will not go into great detail about either of these interfaces, since I don't know nearly enough to write very much about them, but I hope that this is a reasonable introduction.

The next step along this journey was hard work. Once I had failed to find pre-written python code for this sensor online, I had to resort to writing my own. This involved a couple of months of going

through the sensor's datasheet, working out what I needed and what I didn't, and finally writing software to communicate with it.

This task was helped significantly by the python smbus module, that allows me to write and read bytes to a specific device at a specific address (register) on the sensor, in order to set it up in the correct mode, correctly setup and calibrated. My task now was deciding *what* bytes to write where and when.

In the interest of space, I will not document my process in massive detail, but I will give the main parts of the sensor that are used in the project. I will not show the parts that I left out unless relevant to discussion. All being said, the datasheet provided by InvenSense, the manufacturer, is just under 100 pages, most of which I had to read several times to understand the sensor.

A note: this sensor has a "FIFO" (first-in-last-out) volatile memory store, to act as a queue for data to pass to another device. Since I am reading directly from the device, this is not necessary for my uses, and any FIFO options in the datasheet were ignored, as it is disabled by default

### *2.5.3.1: Configuration*

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1A | 26 | CONFIG | R/W | - | FIFO_MODE | EXT_SYNC_SET[2:0] | | DLPF_CFG[2:0] | |
| 1B | 27 | GYRO_CONFIG | R/W | XGYRO_Cten | YGYRO_Cten | ZGYRO_Cten | GYRO_FS_SEL [1:0] | - | FCHOICE_B[1:0] |
| 1C | 28 | ACCEL_CONFIG | R/W | ax_st_en | ay_st_en | az_st_en | ACCEL_FS_SEL[1:0] | - | |
| 1D | 29 | ACCEL_CONFIG 2 | R/W | - | | | | ACCEL_FCHOICE_B | A_DLPF_CFG |

*Figure 22: The configuration registers*

This is where the fun starts. The leftmost column is the register address (hexadecimal). This gives me the place where I can write the data to the device. These four registers are configuration registers.

There are certain things here that I can leave as they are. In register 1A, three EXC_SYNC_SET bits are not used, as they are only for SPI communications. The DLPF (digital low pass filter) is useful to me. It removes high frequency noise from the sensor's data, which is always desirable. The cut-off frequency of this filter is determined by the three least significant bits in this register, in accordance with this table:

| FCHOICE | | DLPF_CFG | Gyroscope | | | Temperature Sensor | |
|---|---|---|---|---|---|---|---|
| <1> | <0> | | Bandwidth (Hz) | Delay (ms) | Fs (kHz) | Bandwidth (Hz) | Delay (ms) |
| x | 0 | x | 8800 | 0.064 | 32 | 4000 | 0.04 |
| 0 | 1 | x | 3600 | 0.11 | 32 | 4000 | 0.04 |
| 1 | 1 | 0 | 250 | 0.97 | 8 | 4000 | 0.04 |
| 1 | 1 | 1 | 184 | 2.9 | 1 | 188 | 1.9 |
| 1 | 1 | 2 | 92 | 3.9 | 1 | 98 | 2.8 |
| 1 | 1 | 3 | 41 | 5.9 | 1 | 42 | 4.8 |
| 1 | 1 | 4 | 20 | 9.9 | 1 | 20 | 8.3 |
| 1 | 1 | 5 | 10 | 17.85 | 1 | 10 | 13.4 |
| 1 | 1 | 6 | 5 | 33.48 | 1 | 5 | 18.6 |
| 1 | 1 | 7 | 3600 | 0.17 | 8 | 4000 | 0.04 |

*Figure 23: The bit configurations for the low pass filter*

The FCHOICE bits can optionally disable the low pass filter altogether, and are set in the 1B GYRO_CONFIG register, in fig. 22.

The accelerometer has its own low-pass filter which is configured in a very similar (almost identical) way.

The next main piece of configuration is in the places marked “GYRO_FS_SEL” and “ACCEL_FS_SEL” in fig. 22. The purpose of these 4 bits is to specify the possible range of the sensor’s output. This is very important. The sensor has 4 options for acceleration: ±2, 4, 8 and 16G and the gyroscope has ±250, 500, 1000 and 2000DPS (degrees per second) options. These must be chosen carefully under two constraints:

1) Not to have the sensor data clip at the range (like distortion on a speaker)
2) Not choose the range to be too large, which would sacrifice accuracy.

The second of these needs some explaining. There are only 16 bits dedicated to each of the output axis (x, y and z for both accelerometer and gyroscope) and so doubling the range halves the precision of the accuracy. For testing, I chose 4G for the accelerometer range and 1000DPS for the gyroscope, as I thought that these fit the two criteria well.

This really only just scratches the surface of what this sensor is capable of, however, most of the other options are just that – optional settings which are only used in niche cases or not useful to my specific use case. This is really all that’s needed to get the sensor up and running with the right setup to read data: the focus of the next section.

#### *2.5.3.2: Reading the Data*

Once the sensor was all up and running, the next task was to start the sens*ing.* This is comparatively simple from a data transfer point of view, but this data must be formatted and converted into a human-readable, control system friendly SI unit.

Note: The MPU-9250 is a MEMS (Micro ElectroMechanical System) sensor. Its working principle is to use tiny structures that change shape when (and only when) the thing being sensed alters, this change in shape results in a change in capacitance to a fixed frame, which is converted into a binary signal. This is the bare-bones working principle behind most MEMS sensing technology, but since I didn’t have anything to do with the design of this sensor a more detailed explanation is far beyond the scope of this report.

The registers containing the sensor data are as follows:

| | | | | |
|---|---|---|---|---|
| 3B | 59 | ACCEL_XOUT_H | R | ACCEL_XOUT_H[15:8] |
| 3C | 60 | ACCEL_XOUT_L | R | ACCEL_XOUT_L[7:0] |
| 3D | 61 | ACCEL_YOUT_H | R | ACCEL_YOUT_H[15:8] |
| 3E | 62 | ACCEL_YOUT_L | R | ACCEL_YOUT_L[7:0] |
| 3F | 63 | ACCEL_ZOUT_H | R | ACCEL_ZOUT_H[15:8] |
| 40 | 64 | ACCEL_ZOUT_L | R | ACCEL_ZOUT_L[7:0] |
| 41 | 65 | TEMP_OUT_H | R | TEMP_OUT_H[15:8] |
| 42 | 66 | TEMP_OUT_L | R | TEMP_OUT_L[7:0] |
| 43 | 67 | GYRO_XOUT_H | R | GYRO_XOUT_H[15:8] |
| 44 | 68 | GYRO_XOUT_L | R | GYRO_XOUT_L[7:0] |
| 45 | 69 | GYRO_YOUT_H | R | GYRO_YOUT_H[15:8] |
| 46 | 70 | GYRO_YOUT_L | R | GYRO_YOUT_L[7:0] |
| 47 | 71 | GYRO_ZOUT_H | R | GYRO_ZOUT_H[15:8] |
| 48 | 72 | GYRO_ZOUT_L | R | GYRO_ZOUT_L[7:0] |

*Figure 24: The data registers*

These data registers are read-only, as shown by the ‘R’ in the 4$^{th}$ column. This sensor has two output registers for each channel of the accelerometer and gyroscope. This is for the purpose of precision. Each register is strictly one byte (eight bits). This means that two registers can express 256 times the

number of values that can be expressed by only one register. The 'H' and 'L' on the register names are standing for the words 'high' and 'low'. Simply, this means that the high register is placed on the left of the low register to form the complete value.

There is one catch to reading this data, however: the values read must be formatted. This has two stages:

1) Formatting from binary two's complement
2) Converting the result into S.I. units

Two's complement is a simple scheme for encoding signed integers in binary. Since the accelerometer or gyroscope data can be negative, the system needs a way to express this in binary. For 16 bits, the full range of the registers is still used, just 'shifted' such that the zero, effectively, lies in the middle of the range:

| | |
|---|---|
| -4 | 100 |
| -3 | 101 |
| -2 | 110 |
| -1 | 111 |
| 0 | 000 |
| 1 | 001 |
| 2 | 010 |
| 3 | 011 |

This table represents three-bit values from -4 to 3. Notice that the positive numbers are represented as usual. As a number goes negative, the value wraps around in the binary system, such that -1 is 111. These values then decrease in traditional binary fashion as they become more negative. This is very simple to convert back into a regular signed number. In this case, a program to do this would:

1) Check if the raw binary was greater than or equal to 4;
2) If so, subtract 8 from it;
3) There you have it: The number is formatted.

To convert into SI units is also not too difficult. Once the actual value of the data has been obtained, it will be between two upper and lower bounds. This upper bound is clearly the maximum value that I have set for the sensor, governed by the FS_SEL bits (fig. 22), and the lower bound the minimum. Once this is done, linear mapping from these bounds onto one another will give the true value.

### 2.5.4: Conclusion

Sensor control, communication and setup can be as much an art as it is a science. The setup of, especially, the MPU-9250 sensor was a great deal of hard work, spread out over the middle months of 2020. In the interest of length, I have omitted some of the details of calibration and coding in this section. The details of this process are not fundamentally important. The important – and very satisfying - thing about this is that once the code has been completed, it *works.* All I have to do now is write a line of code: "mpu9250.read_accel()" and the data will come right back to me: I don't have to work at all.

In the next section, I will present some more hardware: the motors. These were tested outside of the quadcopter assembly before being put in, to be able to get a very good idea of how they behaved in all circumstances before being assembled, where there is ultimately little room for adjustments that might have to be made in retrospect.

## 2.6: Motors

### 2.6.1: Introduction

This section focusses on another integral part of the quadcopter: the motors. Whereas some of the science of motors was presented in 1.3.2, this section will aim to discuss the practical side of actually getting the motors up and running.

Safety is a primary concern here. These motors can spin at up to 28000 rpm or so. With a 5” propeller attached, the tips can spin at up to about 190m/s. That is not funny. That could do some serious damage to somebody or something, so mounting safely is a primary concern.

Having said that, the process of getting the motors going for the first time was great fun, and, if anything, the danger adds a certain level of excitement and anticipation to my experiments.

### 2.6.2: The Setup

At this point in the project, the only hardware I had available to me was a single brushless motor, some ESCs (electronic speed controllers), a battery and propellers. This meant that I was only able to test a single motor on its own, mounted to a thick piece of card stuck down to a stone slab. This setup was very crude, but it allowed me to at least see if my knowledge of the ESCs’ input signal was correct.

*Figure 25: The motor*

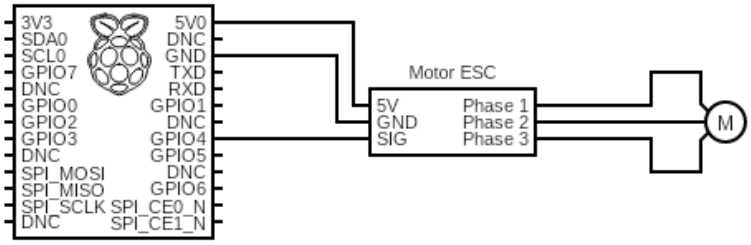



*Figure 26: A schematic of the wiring setup*

Fig. 26 shows the wiring.
The Raspberry Pi and ESC both have their own power supply, the Raspberry Pi’s is from a small USB charger (5V), and the ESC’s is from the LiPo battery (12V). The ESC has a separate 5V logic circuit that takes a control signal for the motor’s speed.

This control signal is a regular pulse, anywhere between 50 and 500Hz. This frequency is not important, but the length of the pulse is. This pulse length can be anywhere from 1-2ms in length: 2ms is 100% throttle and 1ms is 0% throttle. This pulsed technique is part of a system called PWM (pulse width modulation).

This signal can either be generated using software calculated time or physical built-in hardware timers. For the drone, the signal must be very precise (±4µs) otherwise the propeller speed will be very inconsistent. Software PWM is not this precise and so hardware PWM must be used. On the raspberry pi there are only two hardware timers (outputting on pins 4, 12; 5, 13 respectively), and so it is not suitable for controlling four propellers (see 1.4, paragraph 2). However, this system works just fine for one motor.

The ESC has some quirks which I discovered during my research:

- It must be calibrated so that throttle range is exactly between 1 and 2ms;
- To enable the ESC, the throttle must be dropped to 0%.

These represented problems that I had to overcome during testing.

### 2.6.3: Conclusion

After initial problems had been fixed, I had a working knowledge of how to use the motor-ESC combination effectively and what was necessary to get the best performance from it.

As I mentioned, this part of the project was great fun and acted to increase my anticipation and confidence to keep working at it, and to persevere with the next task: verifying the control code.

## 2.7: Verifying the Control Code

### 2.7.1: Introduction

The PID implementation in 2.4 was the first main part of the control system, and the next part was testing this implementation upon my mathematical model developed in 2.3. I decided to delay this section until I had explained the sensors and motors because they – fundamentally – represent both the input and output to this control system, which illustrates nicely how the code and hardware must work seamlessly together. In this section, however, all the sensors and motors will be virtual.

The data that represents them is ideally as close as possible to what it would be in the physical world, so that I get a very good idea as to how the drone will behave when it is given this control data.

This section will look mostly at the development of the thrust PID.

### 2.7.2: The Testing

The testing of this code mainly involves using trial-and error to tune a PID. This process allows me to find out if there are PID weight values that will lead to the desired output from the quadcopter. I am using a python library called matplotlib to display the output from the virtual drone as a graph, which is very useful for analyses.

```
heightData = []
thrustPid = PID()
thrustPid.p_weight = 0
thrustPid.i_weight = 0
thrustPid.d_weight = 0
thrustPid.desired_data = 0.5
```

This code is the parameter setting for the PID controller, and is where all of the adjustments to the controller take place. The rest of the code is simply a loop which updates this controller with new data, and feeds its output back to the drone. Note that for all my tests, the drone is aiming for a height of 0.5m. For the first test, I simply set all weights to zero:

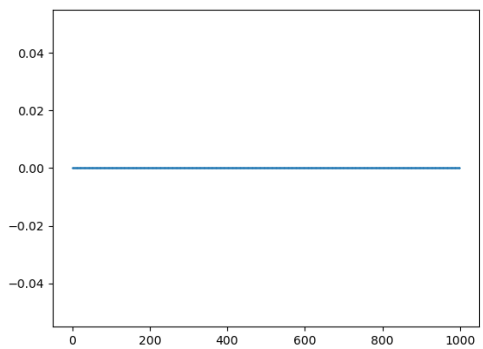



*Figure 27: P=0*

Perfect. This was expected, since the drone can never have any output to the propellers with all weights equal to zero.

The x axis is timesteps, in this case 0.02s, and the y axis is metres. A problem at the beginning of the testing was that the integrator weight had to be exceeding low and the differential weight disproportionately high to make the controller work. This is because the controller didn't take this timestep into account and so, effectively the timestep was always 1s making the slices for integration too large and the gradient too shallow for the differentiation channel. After this was solved, the system worked far more smoothly.

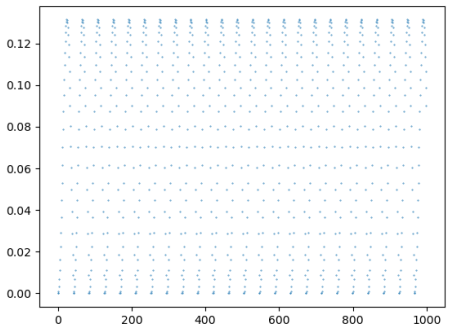



*Figure 28: P=90*

Firstly, I tried tuning the proportional channel. I found that the propellers don't spin fast enough to lift the drone off the ground up unto a weight of 78 or so. At weight 90 this is the output.

Oh dear. The drone gets off the ground, but barely makes it to 0.12m and then falls again. Clearly this is not acceptable. The obvious solution is to increase the weight, and also increase the integrator to stop the quadcopter falling back down.

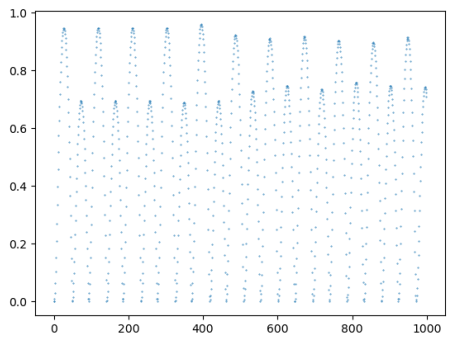



*Figure 29: P=150, I=100*

This is not good either. Even with a large weight for both, the drone is not holding its altitude. This is where the 3rd channel comes into play.

Clearly, the signal is fluctuating too much, even up to twice the amount of the desired output. Therefore, the derivative of the signal is too high; so is must be reduced. The D channel of the controller serves this purpose exactly.

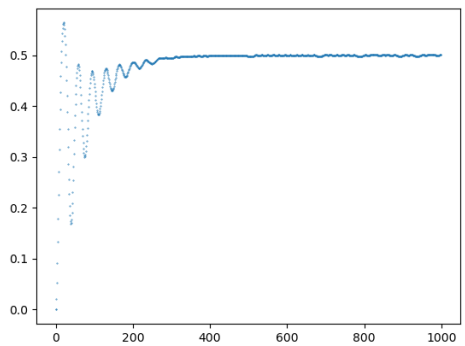



*Figure 30: P=150, I=100, D=8*

Fig. 30 demonstrates the power of doing this. For a D of 8 the output is still fairly messy, but much better. The drone now levels off to a correct value of 0.5m. After some more tuning, I found D=45 to be a well-tuned value:

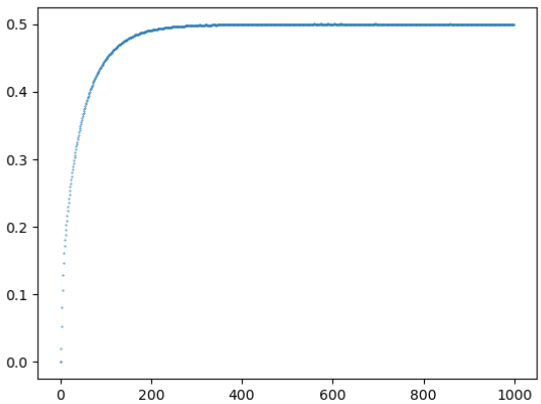



*Figure 31: P=150, I=100, D=45*

This is the final state of this controller. The oscillation has been completely damped out, meaning that there is no overshoot at all. This is what a well-tuned controller's output should look like, and the data found here will almost certainly make its way onto the code that runs live on the drone.

### 2.7.3: Conclusion

This tuning process is far lengthier than it may seem from reading this section. This process was one that I had to teach myself from scratch, and so I took many hours thinking over my thought process, before finally coming to conclusions that lead to the output that I wanted. Having said this, the end of the tuning was that start of a realisation that the control system did indeed function as expected, and so the majority of the hard work had been done.

The next section is, arguably, the most exciting. After Christmas 2020, I finally had all of the components that I needed to assemble the entirety of the drone, and solder all of the electronics: the subject of the next section.

## 2.8: The Quadcopter's Assembly

### 2.8.1: Introduction

In this section, I will present my process of the assembly of the quadcopter, which was in early 2020 after I finally had all of the components available to me. The assembly process involved four main stages:

1) Assembling the frame;
2) Placing the electronics on the frame;
3) Soldering the larger wires;
4) Wiring the smaller wires.

### 2.8.2: The Assembly

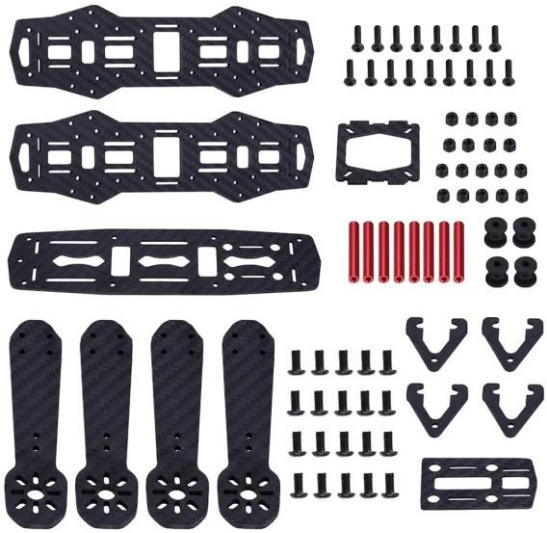


*Figure 32: A layout of the parts of the drone frame, image from the amazon product listing of the frame*

Fig. 32 shows the various screws and plates that constitute the frame for the quadcopter. The frame is completely flat-packable and the plates are built from carbon fibre, such is common in drone frames. See 1.3 for more explanation of this frame.

I began the assembly by screwing the four motors onto their respective mounting arms. The arms of the drone are thicker (3mm) than the main drone plates for strength, and so a specific length of screw must be used to hold the motors in place.

Once this was done, I then attached the main power distribution board to the drone's first base plate. The assembly of the frame is such that there are two plates on the base – the top left two in Fig. 32. These plates sit either side of the motor arms. The power distribution board is square and uses the four central mounting points. Once this was done, I screwed the two base plates together with the motor arms. The final part of this initial setup was to use zip ties to attach the four speed controllers to the base of the drone. This resulted in the following:

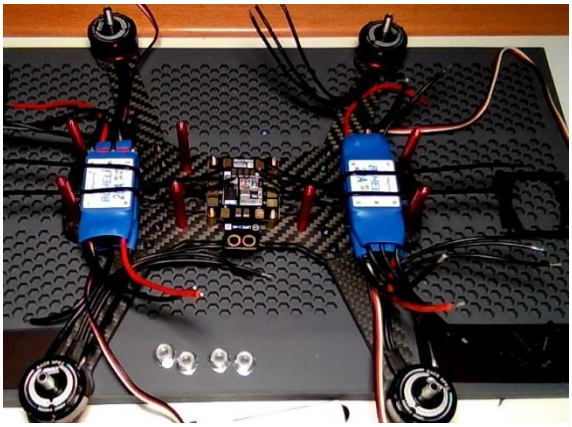


*Figure 33: The initial component layout for the drone*

This resulted in lots of loose wires that all needed attaching to something. This is fairly simple for the motor wires. Once trimmed to length, they were soldered to the respective three wires on the speed controllers. Soldering, at the time, was a fairly new skill to me, and I had to experiment with different solders, temperatures and tips to create a high-quality connection between wires. I eventually found that for larger connections, a temperature around 410°C was best, with a bevel tip. After this, I had to solder the ESC wires to the correct pads on the power distribution board. You will notice that

there are some wires still unaccounted for. Namely, the three header wires coming from each ESC. These are not soldered but connected to the propeller signal outputs from the microcontrollers, which will be done later.

In this figure, I had already attached the aluminium standoffs that secure the upper face of the drone, in preparation for the top plate. The top plate itself has two circuit boards on it: the Raspberry Pi and the Arduino board:

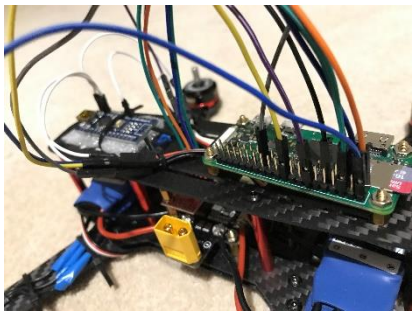


*Figure 34: The quadcopter wiring*

This is figure of this final wiring. The wires look like something of a mess, but the wiring scheme is ultimately simple: the four ESC control wires are routed around to the top of the drone, where the grounds on all four are connected to the Raspberry Pi's ground, via the header pins. The Pi gets its power from the 5V source on one of the ESCs. The four white wires on the Arduino are connected to the signal wires on the ESCs. The raspberry pi is (not in the image) connected to the Arduino by a USB cable, to which it sends motor commands.

As well as this, there was the MPU-9250 sensor mounting to consider. This sensor must be placed at the centre of the drone, and so I chose to use a standoff to mount it to the top of the power board:

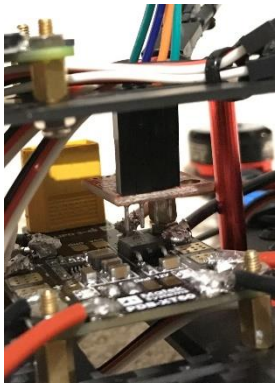


*Figure 35: The MPU-9250 Sensor, mounted*

The wires for the sensor come up through the frame and attach to the Raspberry Pi's $I^2C$ lines, as well as the 3.3V power supply.

This assembly was not complex: the main challenge was the attempt to arrange all of the components according to the constraints of centre of mass, neatness and physical possibility. It was at this point in the project that all of my work had effectively culminated in a single object, which was exciting as well as rewarding.

## 2.9: Controlling the Drone: User Interface

### 2.9.1: Introduction

After I had built the drone, I needed a way of accessing it in order to control it. A possibility for this would be to simply use an off-the-shelf controller which uses standard protocols. This was not a good option, however, since the reception of radio waves would require an extra receiver. Also, the buttons on a controller typically do not provide the variety and detail of control that I would need for the quadcopter. That is why I chose to use Wi-Fi, coupling my laptop – which would run a GUI – with the receiver chip on board the raspberry pi zero W.

### 2.9.2: The Protocol

Since I was not using the standard radio protocol, I settled on using TCP for the Wi-Fi connection between computer and drone. TCP (transmission control protocol), is the main standard used for data transmission over the internet. However, this does not answer the main question: how are bytes of data sent such that they correspond to individual, specific, instructions that the quadcopter must receive and process?

After some thought, I decided that the best way to implement this would be to have a specific syntax for each instruction which conformed to a universal grammar that applied for every instruction. I the

spent half an hour or so coming up with this syntax and, ultimately, an instruction set. Here is a tabulation of the result:

```
To flag the beginning of a message: #

OWS: Only When Stationary
:= and / or selection
;= or selection

---------- Instruction Set ------------|
                                       |
STOP                                   | Stops props with immediate effect.
ARM                                    | Only at boot, to arm quad. OWS
REBOOT                                 |
                                       |
START                                  | To take off. OWS
LAND                                   | To land in a graceful fashion.
                                       |
PID <P;I;D> <t;r;p;y> <abs;rel> <value>| PID trimming command
ALT <abs;rel> <value>                  | To move to a new height
ANG <x;y> <abs;rel> <value>            | Change angular position of the quad
                                       |
START_PID                              | To start the control system.
STOP_PID                               | To stop the control system for raw access. OWS
                                       |
RAW <1:2:3:4> <value>                  | Raw control to all props (ms values)
```

*Figure 36: The instruction set*

This figure is fairly self-explanatory.

This relatively simple set of instructions allows the user of the controller to do almost anything with the quadcopter, with a small set of instructions, making them surprisingly powerful.

Although it would work, my plan was not to type out each individual instruction when I wanted to use it. That would be tedious as well as dangerous: what if I want to stop the drone's propellers at a moment's notice. Typing "#STOP" takes too long to prevent potential disaster. My plan was to create a GUI to control the drone using graphical buttons and sliders, as opposed to raw text.

Here I was faced with the question: how to write the GUI? I have experience with various python graphics libraries, the main one being Pygame. I could have easily chosen this; however, this library does not provide in-built features such as buttons and sliders, which I would need to use. This means that I would have to make them all manually, which would be very time consuming. Instead, I chose to use Tkinter, which is a graphics library that does have these inbuilt features, and saves a lot of time. Since the process of writing a GUI is ultimately repetitive, I will not present any code or implementation details, except to say that each slider or button etc. is tied to a command that is sent to the drone upon interaction. Here is the final result:

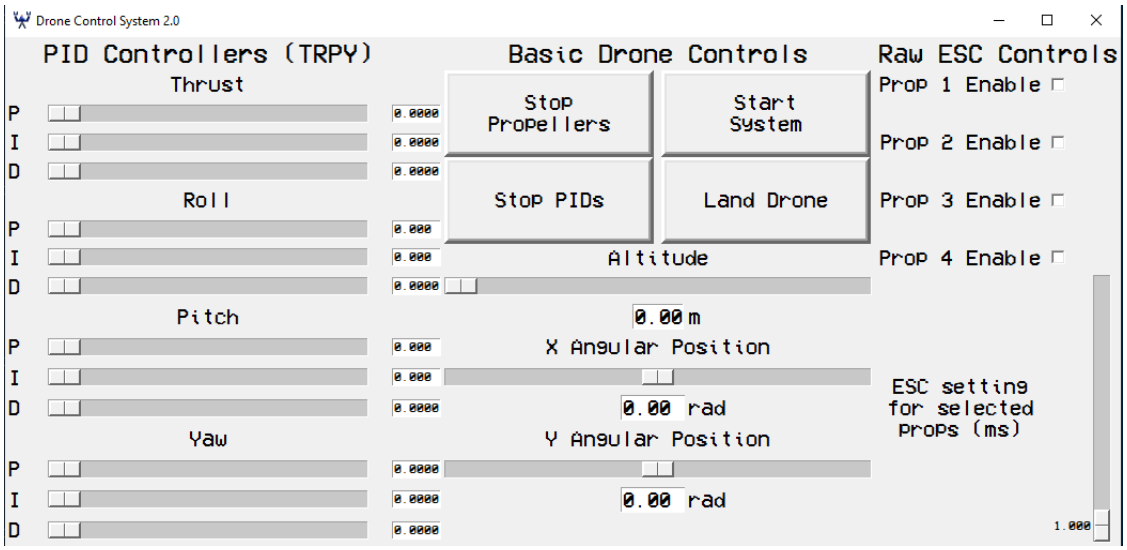



*Figure 37: The final state of the GUI*

### 2.9.3: Conclusion

After this GUI was complete, I was able to have complete control over the drone at a distance, just using the Wi-Fi network available. The side of the story I haven't presented here is the side of the raspberry pi, which has to receive the commands and process them, resulting in the correct output that the command was expressing. This job was not too difficult however, and once in place it is a fairly simple control loop that takes the command, splits it into its constituent parts, and sends necessary commands to the motors as described in 2.6.3.

The next stage in the project is ultimately the final one. At this point everything is coming together: we have,

1) The physical assembly of the drone; (2.8)
2) The ability to sense the drone's physical state and use that data; (2.5)
3) The control system, applied via 4 PIDs to thrust, roll, pitch and yaw; (2.4, 2.7)
4) The ability to send motor actuation commands to the Arduino; (2.6.3)
5) The coupling of all of these systems in a complete control loop.

So, all we need to do now is tune the PIDs, and get the drone in the sky.

## 2.10: Tuning and Data Gathering

### 2.10.1: Introduction

In this section the ultimate goal was to apply what I had learnt in 2.7 and apply it to the physical drone. This comes with an extra consideration. Namely, obtaining the physical properties of the drone such that I could tune the PIDs using the previously described mathematical model (2.3). This data gathering was done in some different and possibly creative ways which I will present in this section of the report.

After data had been gathered, the next step was to overcome the problem of making the mathematical model behave exactly like the quadcopter would in the real world, so that the results I obtained during tuning were similar to the values I should use in testing.

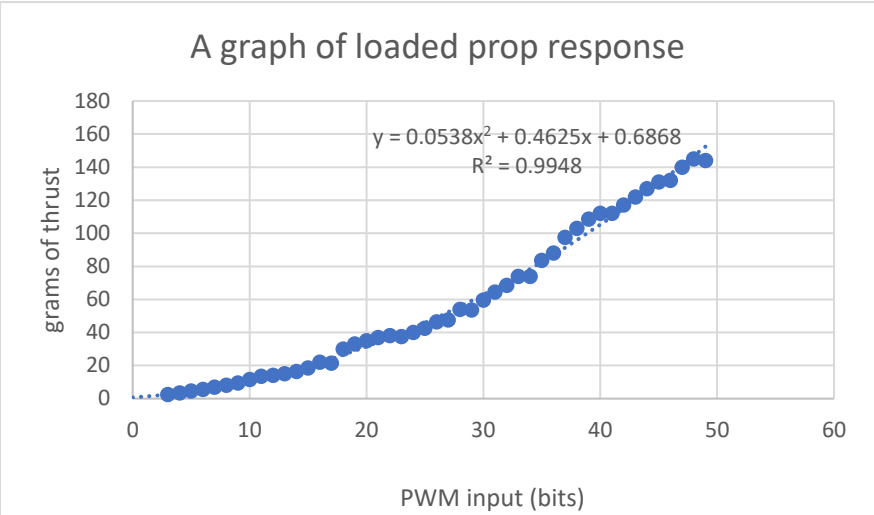
### 2.10.2: The Data Gathering

The first data I had to gather were for the revolution speed and thrust of the motors at different throttle inputs. This is really the most useful set of data I needed since once these are known, the rest of the behaviour of the drone follows naturally.

The main question was how to gather such data. I came up with two approaches:

1) Tying the quadcopter to some scales;
2) Using an improvised stroboscope.

Since I came up with the less useful stroboscope method first, I ended up using both to gather both rotation rate data as well as thrust data. The setup for the thrust data was to use a large stone to strap the drone to some scales, with the propellers facing downwards, for positive thrust readings. I then set the propellers to the desired throttle value between 0 and 100, and read the value from the scales once it had settled. After 50 repetitions (I only carried out the experiment across half the throttle range) I had a set of data:




I only carried out the experiment over this range since above a certain throttle, the flow over the propellers starts to separate, because of the nearby floor surface. This causes inefficient transfer of energy to the air and so anomalous results result.

The results with the strobe are very analogous and lead to the same function, however, the thrust data cannot be directly determined from this data making it less useful to me: the first method was realised after some thought.

After this, the next task was to determine how the force of the propellers would affect the movement of the drone. This is done by finding the mass and mass moments of inertia for the drone. The mass moment of inertia was a problem, since I could not think of a safe experimental way to figure out the moment of inertia. My solution to this problem was not ideal, but it does work.

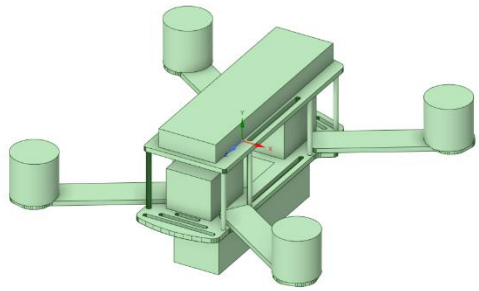


*Figure 38: My CAD model. The blocks represent the different masses that are positioned around the drone. Derived from thing 1271762 on thingiverse.*

I used a rough CAD model of the drone and used the software to extract the moments of inertia, given the material, from the drone, leading to numbers that I could enter into the mathematical model. In order to get the whole system to behave properly, I had to use the function obtained in the above graph to get the force from each propeller given its throttle % at each timestep.

### 2.10.3: Tuning the Quadcopter

This is the last and current stage of the project's progress. Using the data gathered in 2.10.2, I was able to begin testing the resulting values found in the mathematical model on the drone itself. I will not discuss any of the tuning of the PIDs in this model: that was the purpose of 2.7, however I will provide an account of my time tuning the PIDs on the physical quadcopter.

In order to tune the quadcopter, I had to have a setup that was able to rapidly change the quadcopter's parameters on the fly so that I could test a PID setup and retune after seeing the results on a moment's notice, or simply see how a response varies as a function of a parameter. It is important to note that the results obtained using the mathematical model (as in 2.7) after CAD analysis represent only starting points for my tuning efforts, and hence allowed me to set the ranges of the sliders on the GUI to reasonable intervals. However, due to the fact that no model is perfect, I mostly had to apply similar tuning strategies to the physical system as I did to the model to find values that worked well. A problem that I faced during this process was the issue of trying to make the centre of gravity of the quadcopter completely physically central, by moving the battery. This solution, along with tuning the trim of the drone in software, was the main one that I employed during this process: tuning with a poor physical setup is worthless and will give bad results.

Once I had properly placed the centre of gravity, I then began to tune the PID gains one by one. The process, for roll and pitch was as follows, starting on the ground without taking off:

1. Increase the P gain until obvious oscillations were present in the quadcopter (then lower slightly);
2. Increase the D gain to compensate;
3. Repeat 1-2 until a reasonable output is achieved;
4. Adjust I as necessary to stabilise the output and tighten it around its desired value.

This process allows for a very rapid prototype of the control system to be made, which can quickly be changed to alter the quadcopter's behaviour.

The process is very similar for yaw, although it is not necessary to tune yaw, nor thrust to get the quadcopter in the air, which was my main goal. This tuning process is very difficult to illustrate in pictures, so I attach with this report some footage of the tuning process:

Video 1: When P is too high, violent, low frequency oscillations occur

Video 2: When D is too high, small, high frequency oscillations occur

Video 3: When they are tuned correctly, there are very few oscillations, and the quadcopter lifts fairly smoothly into the air. In this clip, the trim needs adjusting since the quadcopter goes off in one direction in the air.

## 2.11: Project Conclusion

This project has been undertaken over the course of about 18 months. The point to which I have developed the project to in this report represents a beginning, not an end. However, I feel that I have learnt a great deal over the course of this time, specifically some basic aspects of control of dynamic systems and the use of code onboard an embedded system, including how this code interacts with sensors, as well as mathematical modelling and efficient use of time. As mentioned in section 1.2, quadcopter and autonomous systems in general are vitally important in modern society. The knowledge of how to use them effectively and safely is therefore invaluable for any engineer or developer, no matter what their field. This is because feedback control works best and only properly when the system to which it is applied has been designed with control in mind, hence engineers of control have to work with a whole spectrum of other engineering disciplines.

It was my choice entirely to take on the task of designing and implementing the control system for the drone as well as building it. It was always an option simply to buy a standard control board and attach it to the ESCs as necessary, using a radio transmitter for control. This would have shaved countless hours off the project's progress, and would have sped the process of getting the quadcopter into the air very substantially. Also, the tried and tested methods which these systems use would have probably improved the performance of the quadcopter, as well as reducing the clutter provided by necessary wires on board the drone. But where is the fun – and, importantly, the learning – in that? Despite this, I do believe that most of the design decisions, especially that of the components for the drone, were suitable.

As with any project there are loose ends or poor decisions that could be modified or improved:

1) Inefficient code;
2) Messy wiring;
3) Leading to poor connectivity;
4) Use of Wi-Fi over Bluetooth;

5) Approximations in the control system.

I will address each of these, and suggest a plausible solution which I could implement in the future.

1) As far as inefficient code is concerned, I would like to write some more of the system's code in the C++ programming language. Since this is a compiled language, it is vastly more efficient than python (sometimes even a whole order of magnitude). The main advantage here is smoother control given by more efficient updating of the drone's control system.
2) The messy wiring leading to poor connectivity is essentially a product of my haste to engage testing of the quadcopter.
3) A far better alternative would be to have a custom PCB fabricated to hold all of the components together, such that the connectivity is physically far more robust, resulting in less bad data and ultimately runtime IO errors in code.
4) In the control of this drone, Wi-Fi is not always applicable because of the possible lack of a nearby network. A far more useful solution is to construct the drone's communication code around Bluetooth or a similar protocol, which is designed specifically for connection between two lone devices, rather than many.
5) Since my knowledge of control theory is not sophisticated enough, I had not yet come across (in thought or reading) a way to allow the thrust PID to take into account that it does not act only in the upwards direction, but also sideways.

Finally, I would like to thank the reader for taking the time to read this report. This report represents one of the achievements that I am most proud of and I hope that there will be more progress upon this project, and others, in the future.

*"Once you have tasted flight, you will forever walk the earth with your eyes turned upward, for there you have been, and there you will always long to return." – Leonardo da Vinci*